Rapport de recherche : Bibliothèque ByzFL

Industrialisation par JSON, Automatisation Colab et Benchmarks de Performance GPU

ABBES Samy & AREZKI Célia

Semaine du 18 Mai 2026

1 Introduction

Suite aux directives issues de la réunion du 22/05/2026, cette semaine a été marquée par une refonte structurelle complète de notre architecture de code. Nous avons abandonné la duplication de scripts au profit d'un système entièrement piloté par des configurations JSON, mis en place un protocole d'exécution industrialisé sur Google Colab avec sauvegarde sur Etulab, et mené nos premiers tests comparatifs de performance entre l'accélération matérielle GPU et l'exécution CPU.

2 Refonte de l'Architecture et Protocole d'Exécution

2.1 Réorganisation du dépôt de fond en comble

Afin de fluidifier le développement et de standardiser nos pipelines d'expérimentations, le code source a été intégralement restructuré. Nous avons isolé les rôles de chaque script et centralisé les hyperparamètres :

- **Gestion des configurations** : Création du répertoire `configurations_experiences/`. Les paramètres globaux partagés sont définis dans `commun.json`, tandis que les fichiers de configurations spécifiques aux scénarios sont regroupés dans le sous-répertoire `scenarios/`.
- **Indexation dynamique** : Mise en place du fichier `scenario_index.json`. Compte tenu de la multiplication des scénarios à tester, ce fichier sert de commutateur central pour spécifier précisément quels scénarios exécuter lors de l'exécution d'une expérience.
- **Scripting applicatif** : Développement du script principal `lancer_experiences.py`, qui charge les fichiers JSON, orchestre l'apprentissage fédéré et exporte automatiquement l'évolution de l'accuracy de chaque round dans des fichiers au format `.csv`.
- **Analyse graphique** : Intégration du script `comparer_experiences.py`, chargé de lire les fichiers CSV générés et de tracer des courbes de convergence comparatives.

2.2 Industrialisation et traçabilité sur Google Colab

Nous avons formalisé un protocole d'exécution résumé dans le fichier `commandes_ggl_colab.txt`. Ce fichier consigne l'ensemble des commandes nécessaires pour :

1. Configurer l'identité Git locale et cloner le dépôt Etulab via un jeton d'accès sécurisé.
2. Installer les dépendances du projet via `uv` et lancer l'exécution du script principal de simulation.
3. Récupérer les éventuelles modifications distantes de l'équipe via un `git pull` juste avant le push, au cas où des fichiers auraient été mis à jour sur le dépôt pendant le temps d'exécution de l'expérience.
4. Exécuter un `git push` manuel et sécurisé en fin de script pour synchroniser et sauvegarder les nouveaux résultats CSV ainsi que les graphiques directement sur `Etulab`.

3 Benchmark de Performance : CPU vs GPU (T4)

Afin de valider l'apport de l'accélération matérielle demandée par les encadrants, nous avons configuré une expérience de référence sur un jeu de données standard (MNIST), étalée sur une durée de **1000 rounds**.

3.1 Périmètre du benchmark

L'expérience croise 4 attaques fondamentales (*SignFlipping, IPM, ALIE, Gaussian*) et 4 agrégateurs (*TrMean, Average, Median, Krum*), combinés à travers 5 scénarios de référence lancés chacun sur 1000 rounds :

— `clean_average` (Baseline saine sans attaque)

— `signflip_average` (Attaque SignFlipping sans défense)

— `signflip_median` (Attaque SignFlipping mitigée par la Médiane)

— `ipm_trmean` (Attaque Inner Product Manipulation mitigée par la Moyenne Tronquée)

— `alie_krum` (Attaque A Little Is Enough mitigée par Krum)

3.2 Analyse comparative des temps d'exécution

L'infrastructure matérielle fournie par Google Colab a été sollicitée sous deux modes distincts pour faire tourner cette suite de simulations :

— **Mode CPU** : Exécution sur un processeur virtuel *Intel Xeon* (2 cœurs, fréquence d'environ 2.20 GHz).

— **Mode GPU** : Accélération via une carte graphique *NVIDIA Tesla T4* (16 Go de VRAM dédiée).

Infrastructure de calcul Colab	**Temps d'exécution total (1000 rounds)**	**Facteur d'accélération**
CPU Virtuel (Intel Xeon, 2vCPUs)	153 secondes	1.0× (Référence)
GPU Matériel (NVIDIA Tesla T4)	**64 secondes**	**2.39**×

Table 1 – Mesure d'efficacité temporelle du pipeline d'apprentissage robuste

Les résultats démontrent une réduction drastique du temps de calcul de plus de **58%** lors du passage sur GPU. Ce gain d'efficacité confirme la nécessité d'exploiter le calcul parallèle sur carte graphique pour exécuter les futures campagnes de tests à grande échelle, impliquant des distributions non-IID via Dirichlet ainsi que des variations systématiques de graines aléatoires (*seeds*).

4 Conclusion et Perspectives

L'infrastructure de notre projet est désormais industrialisée, robuste et prête pour des exécutions massives. Les hyperparamètres critiques sont isolés dans des fichiers JSON et les résultats sont traçables. La prochaine étape consistera à lancer l'ensemble des scénarios indexés et à exploiter le script `comparer_experiences.py` pour analyser la résilience des agrégateurs lorsque la proportion de clients byzantins varie.